ALU - 4bit

a, b, y, carry, op, flag, overflow, cuuneq, zero :ports

Add

Sub

oR

AND

XOR

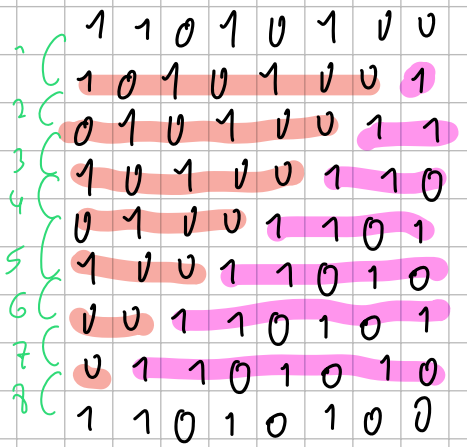
Shift (left, right)

not

zero : y == 4'b0000

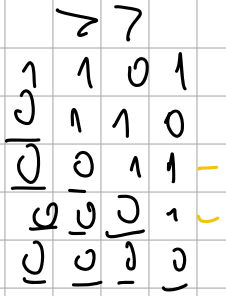
overflow : (a[3] == b[3]) && (a[3] != y[3]) // add

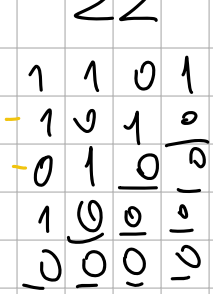
overflow : (a[3] != b[3]) && (a[3] != y[3]) sub

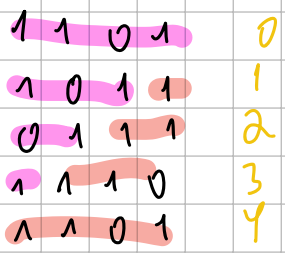


{a[w-1:i:0], a[w-i]}

{a[w-3:0] a[w-1 : w-2]}



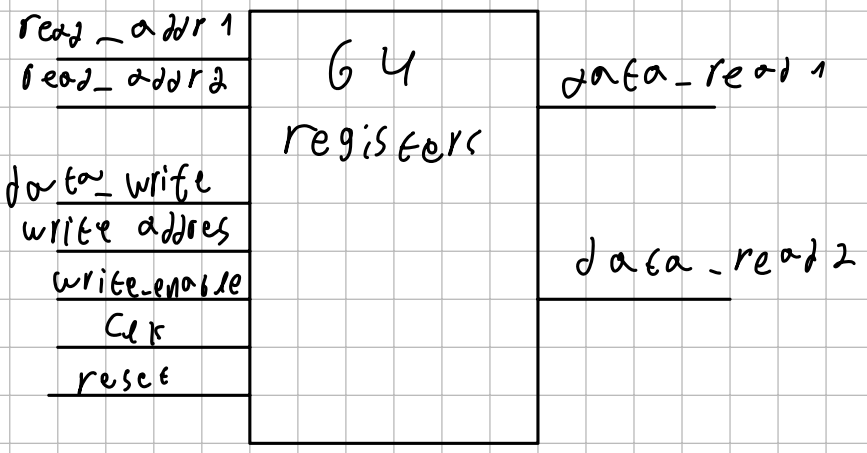




Register File

number of register 64

Size of register 4



input read_adr1, read_addr2, write_addr

input clr, write_enable

input data_write

output data_read1, data_read2

$address \in \{0 \dots 63\} \rightarrow \lceil \log_2 63 \rceil = 6$

address size 6 bits

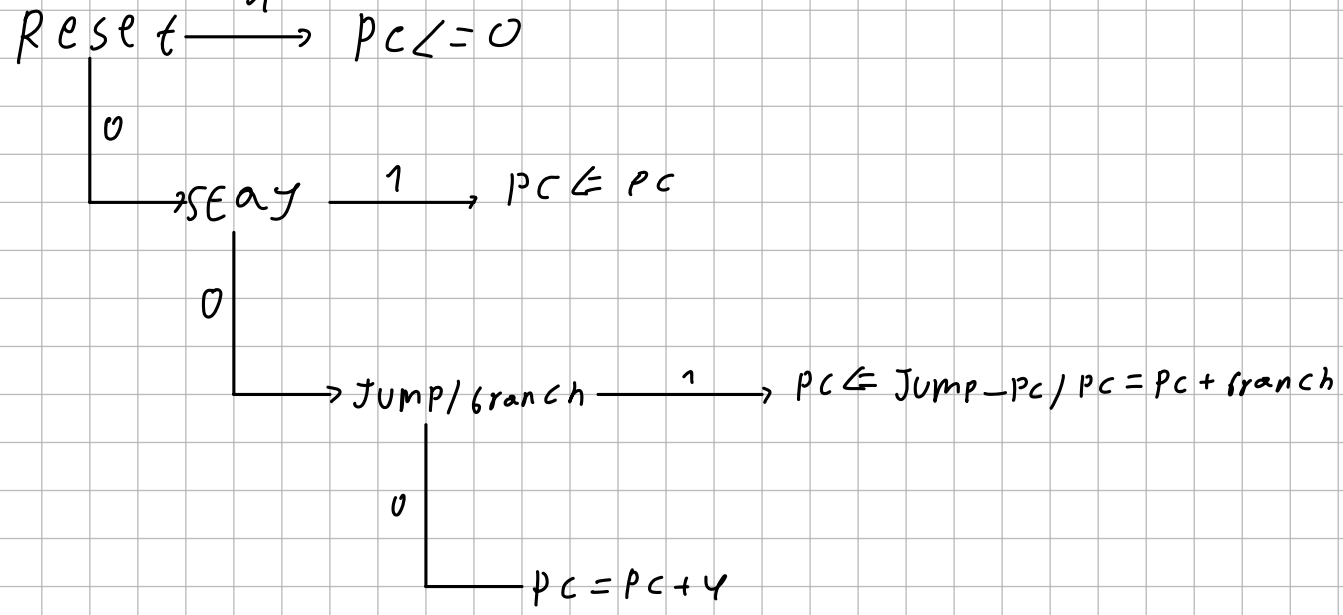
מכיוון שכל ב.ר שומר קצת קטן יותר PC

pc = pc+4

reset pc=0 → mux: next_pc<=□

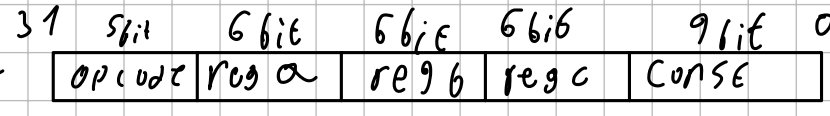
branch pc = pc + jump

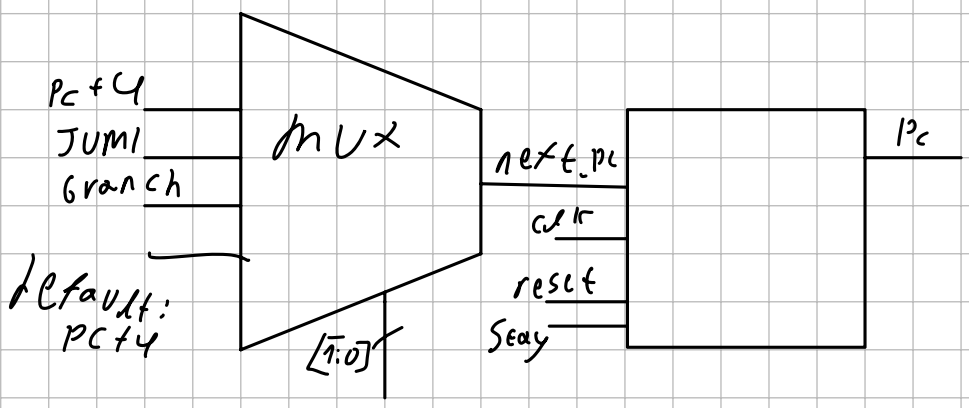
pc = pc

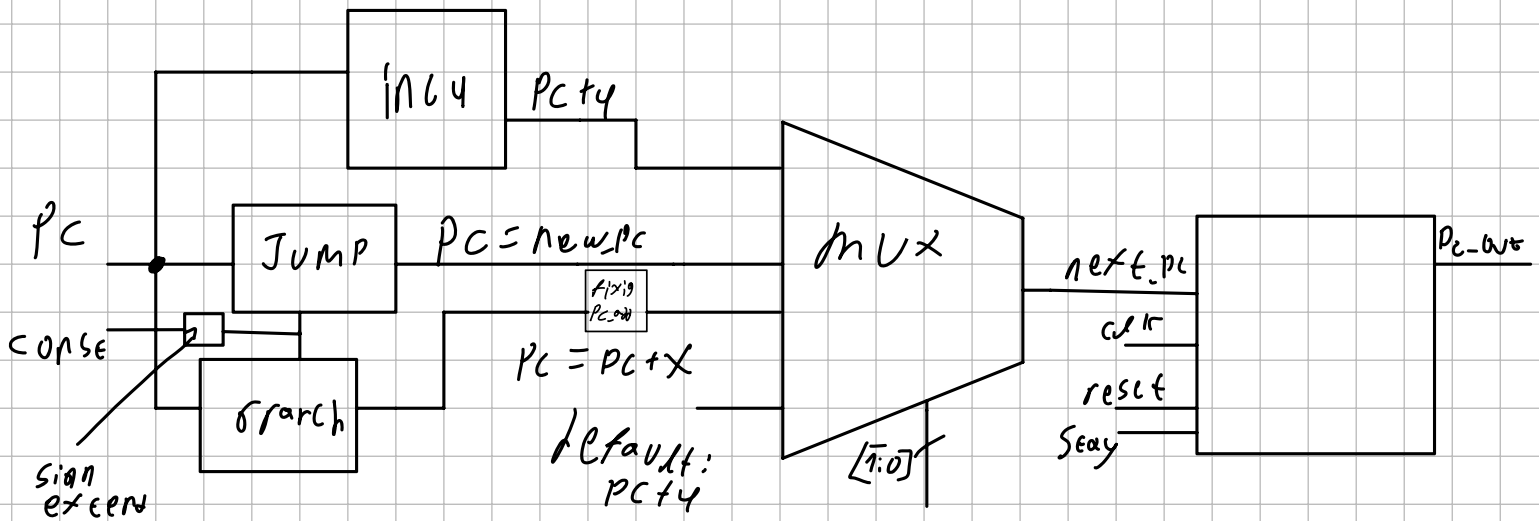


PC_width 16 bit

intruction_width 32 bit



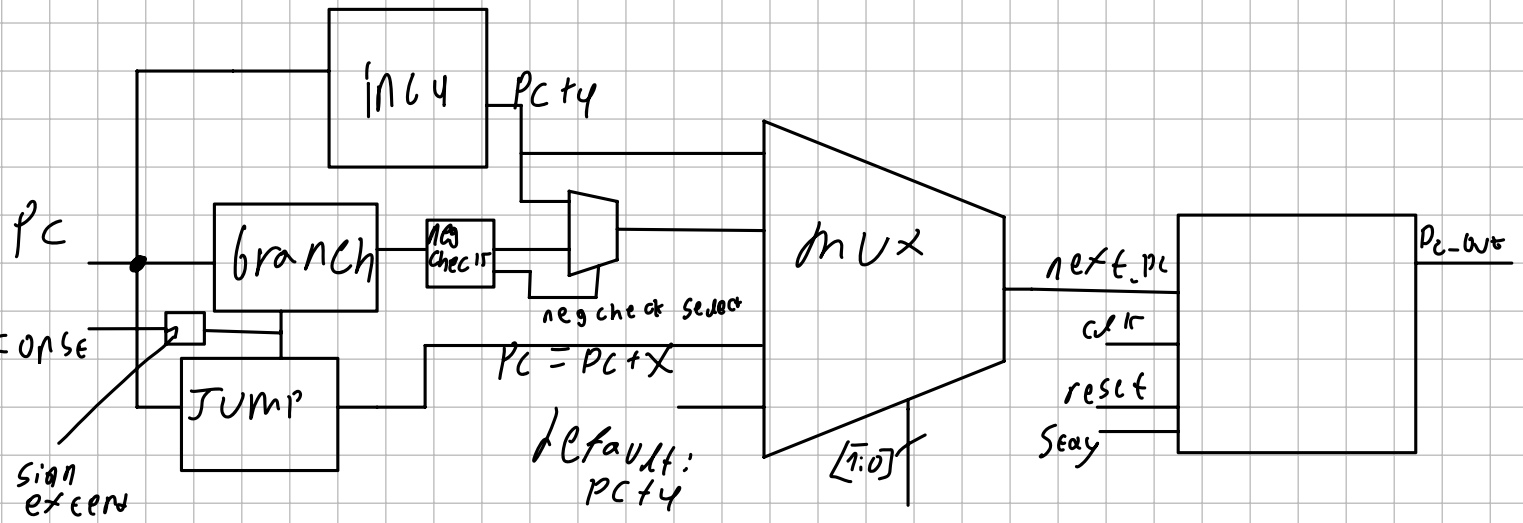


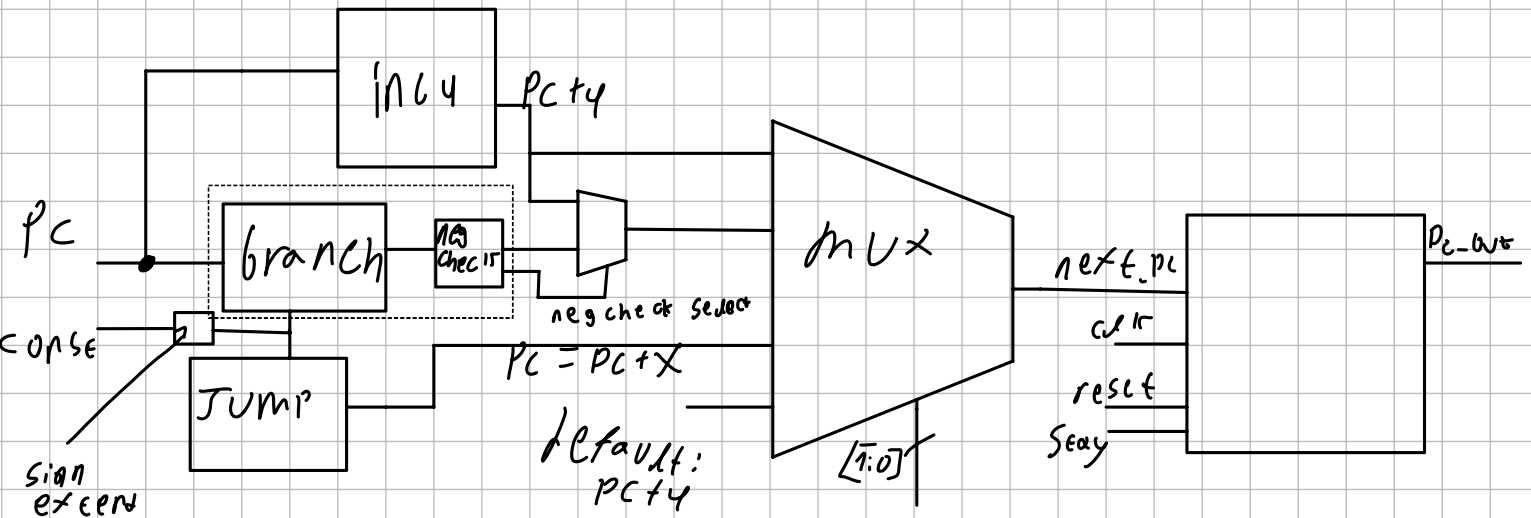


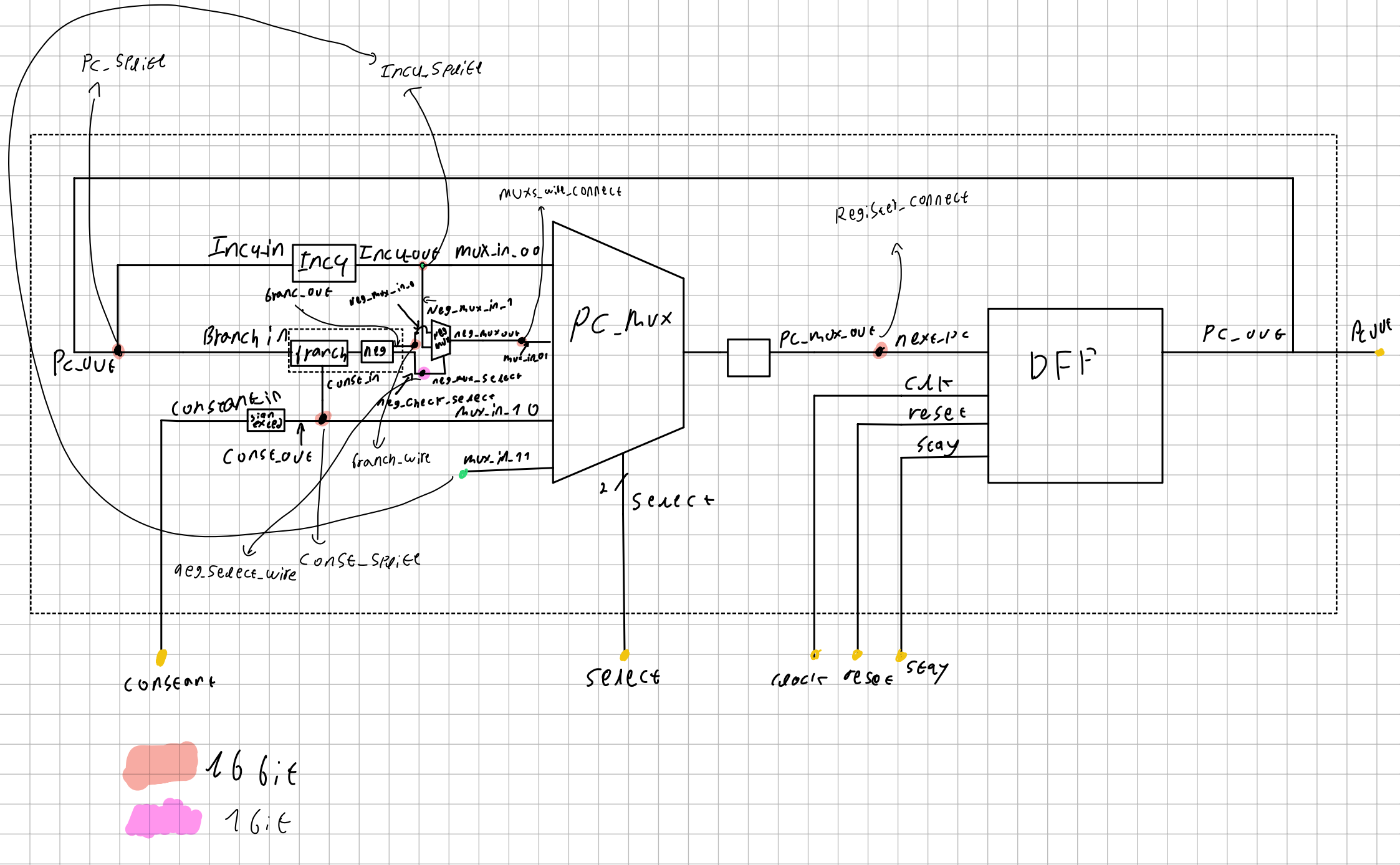
	modules	pore
Inc 4	☑	☐
sign exeension	☑	☐
Jump	☑	☐
Branch	☑	☐
mux	☐	☐
fixed PC	☐	☐
PC Regiseer	☐	☐
main pc	☐	☐

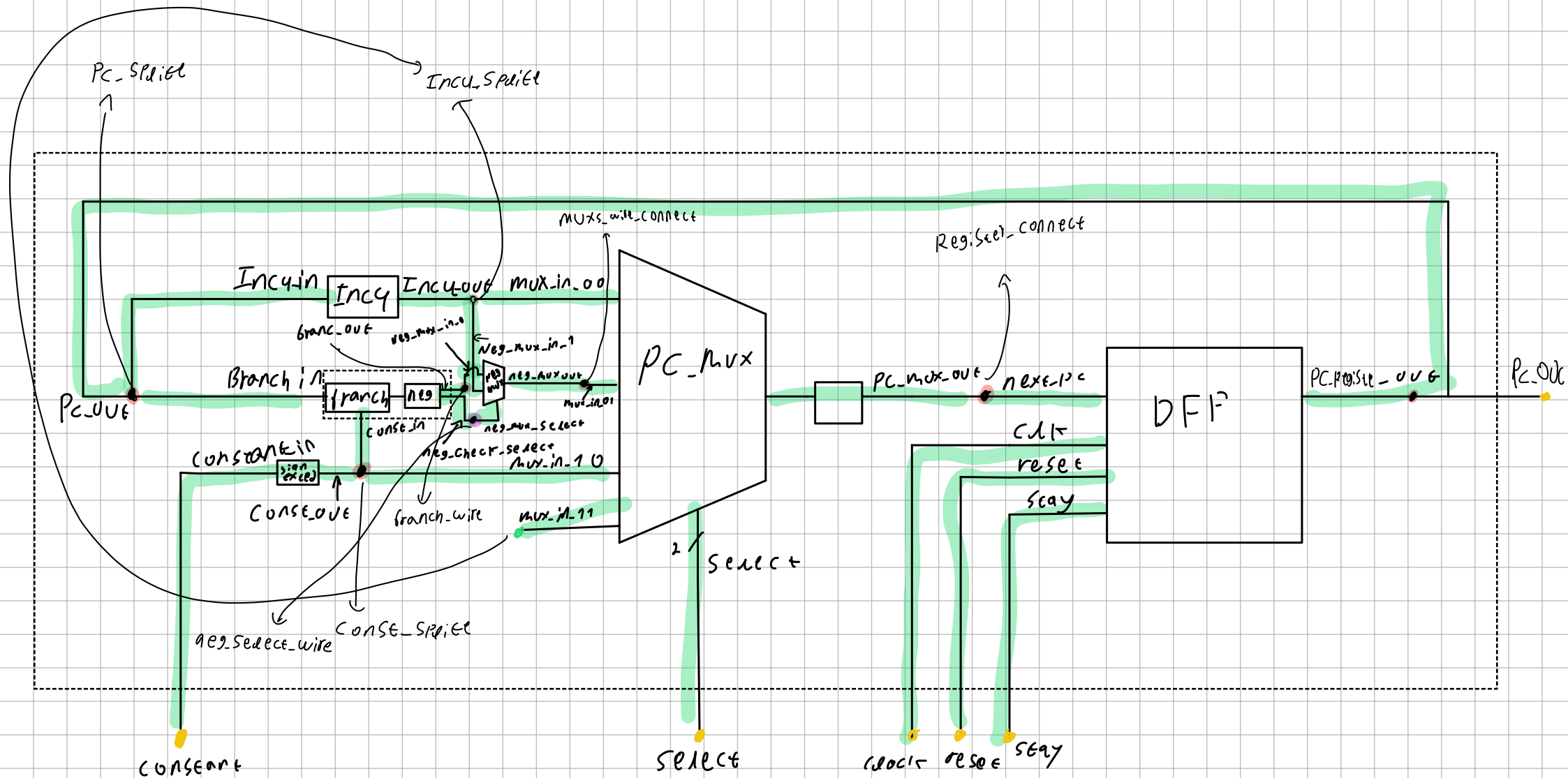
mudy check ==26'00

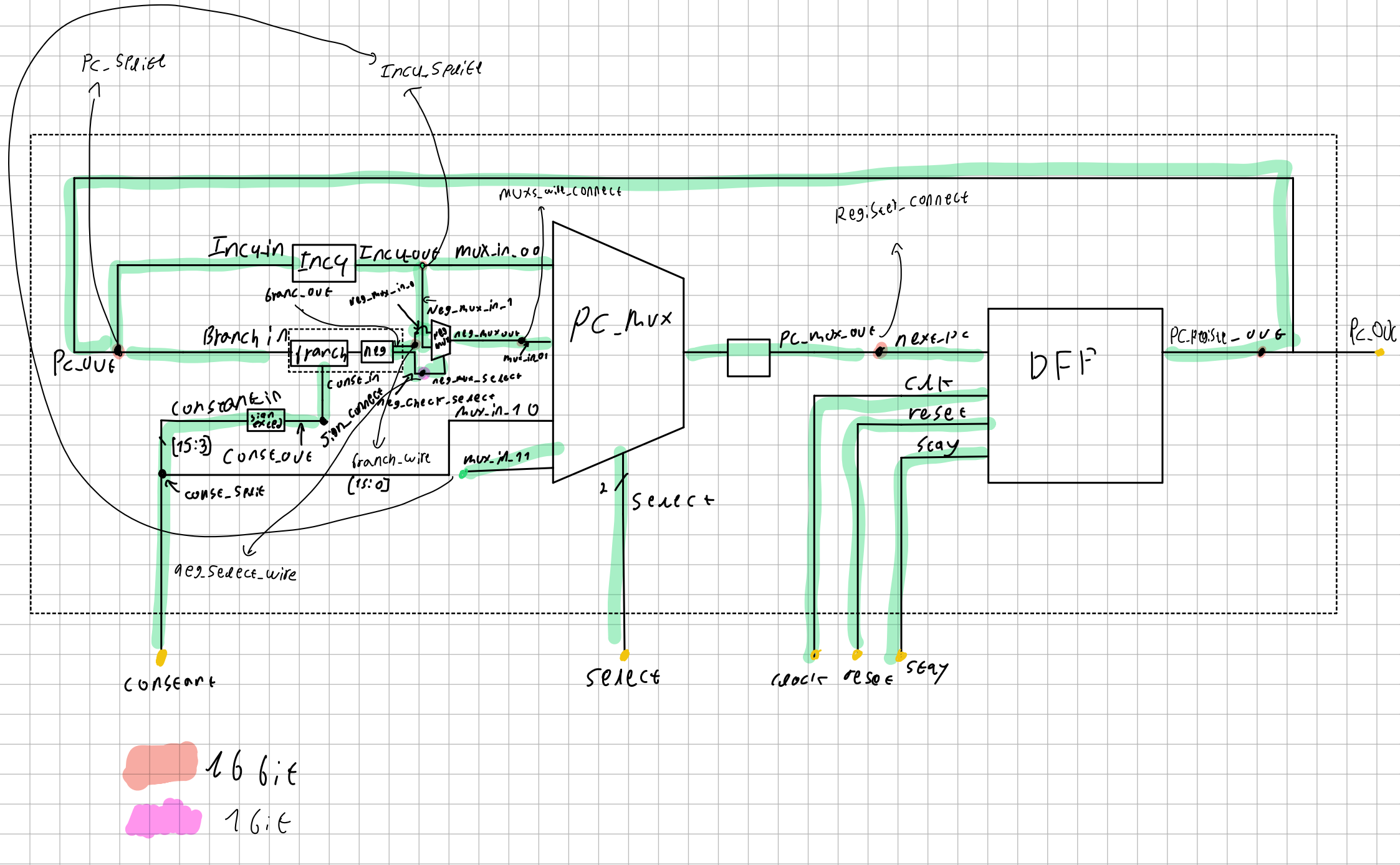
neg check [] == 76'1

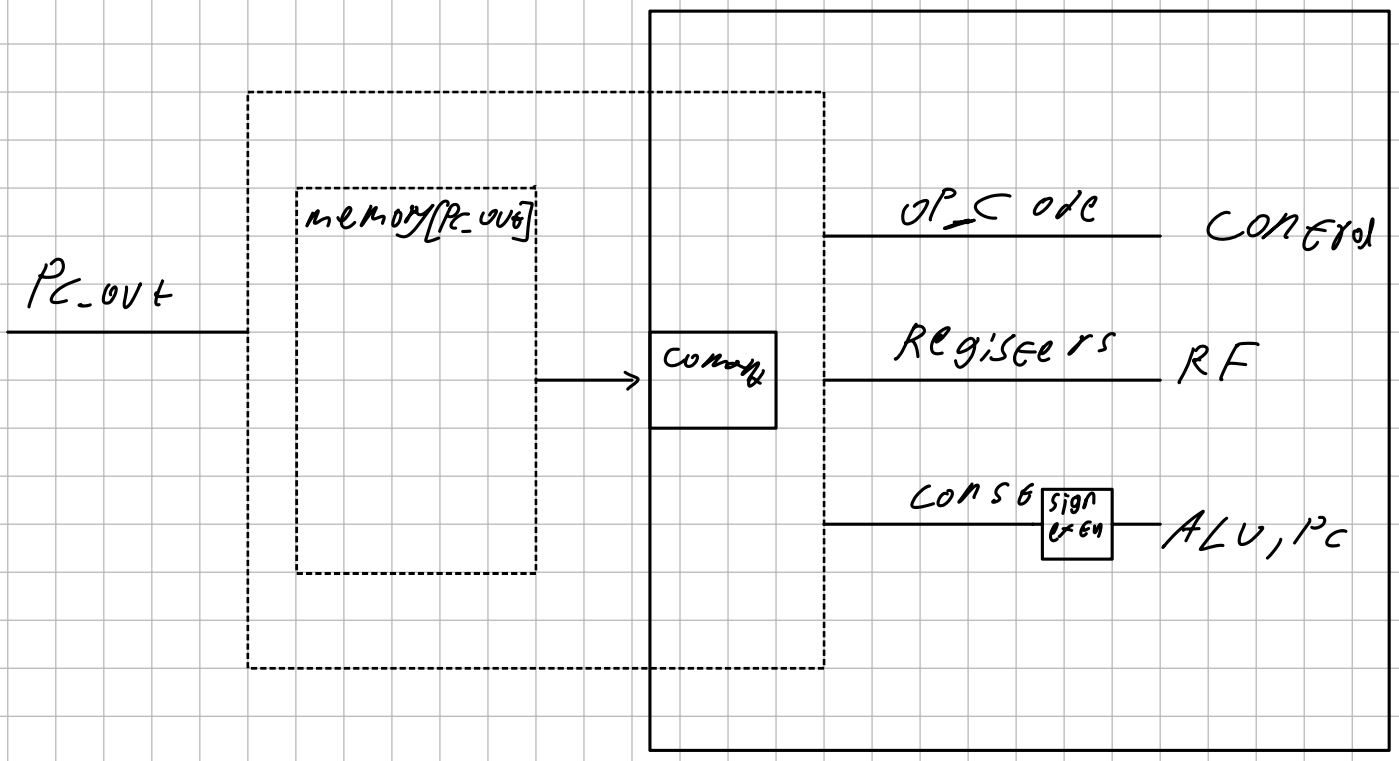

inc4
PC+4
PC
branch
neg check
neg check select
PC = PC+X
default: PC+4
MUX
[1:0]
next_pc
clk
reset
Stay
Pc_out
sign extend

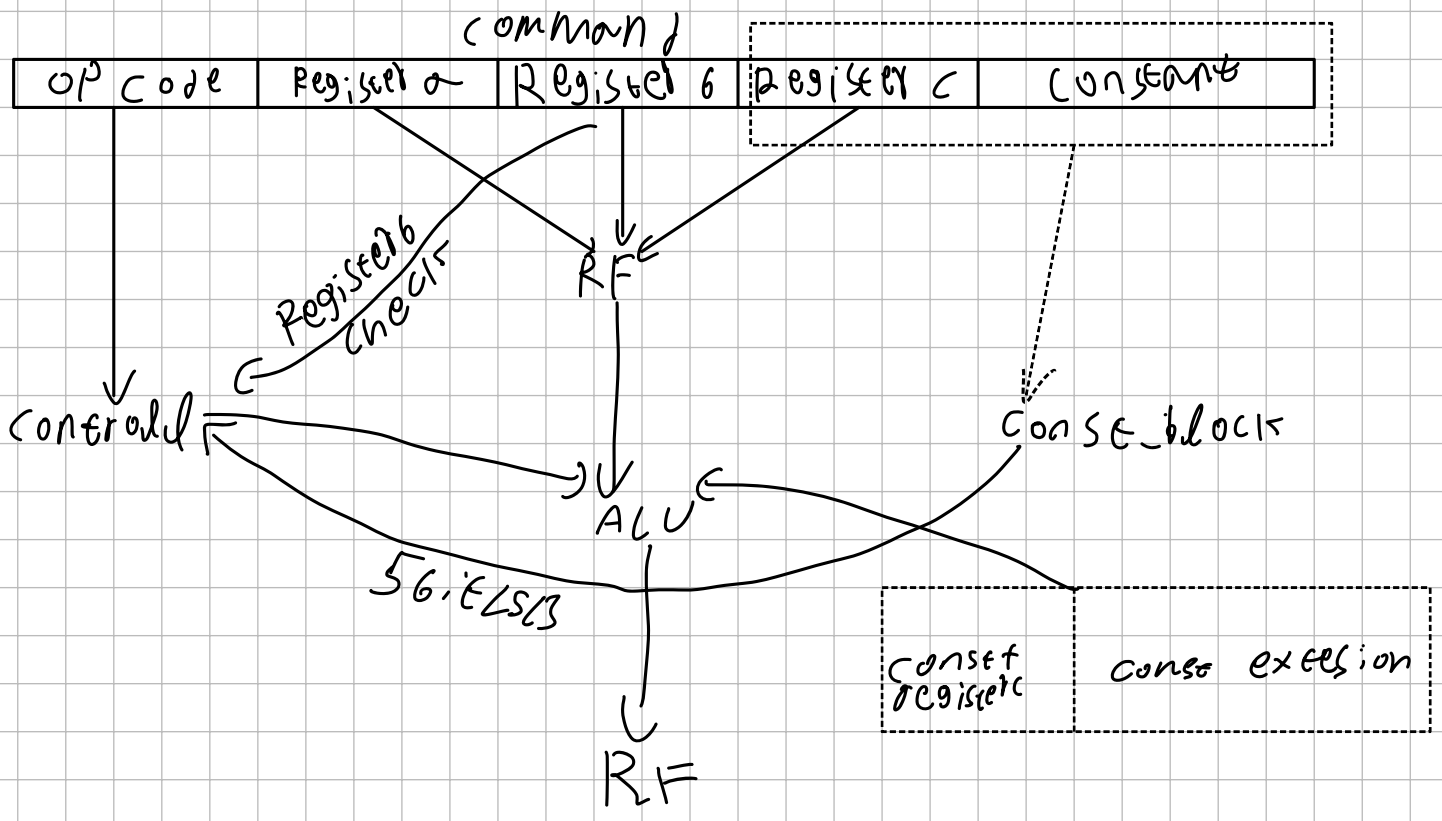

inc4
PC+4
PC
branch
neg check
neg check select
PC = PC+X
default: PC+4
MUX
[1:0]
next_pc
clk
reset
Stay
Pc_out
sign extend

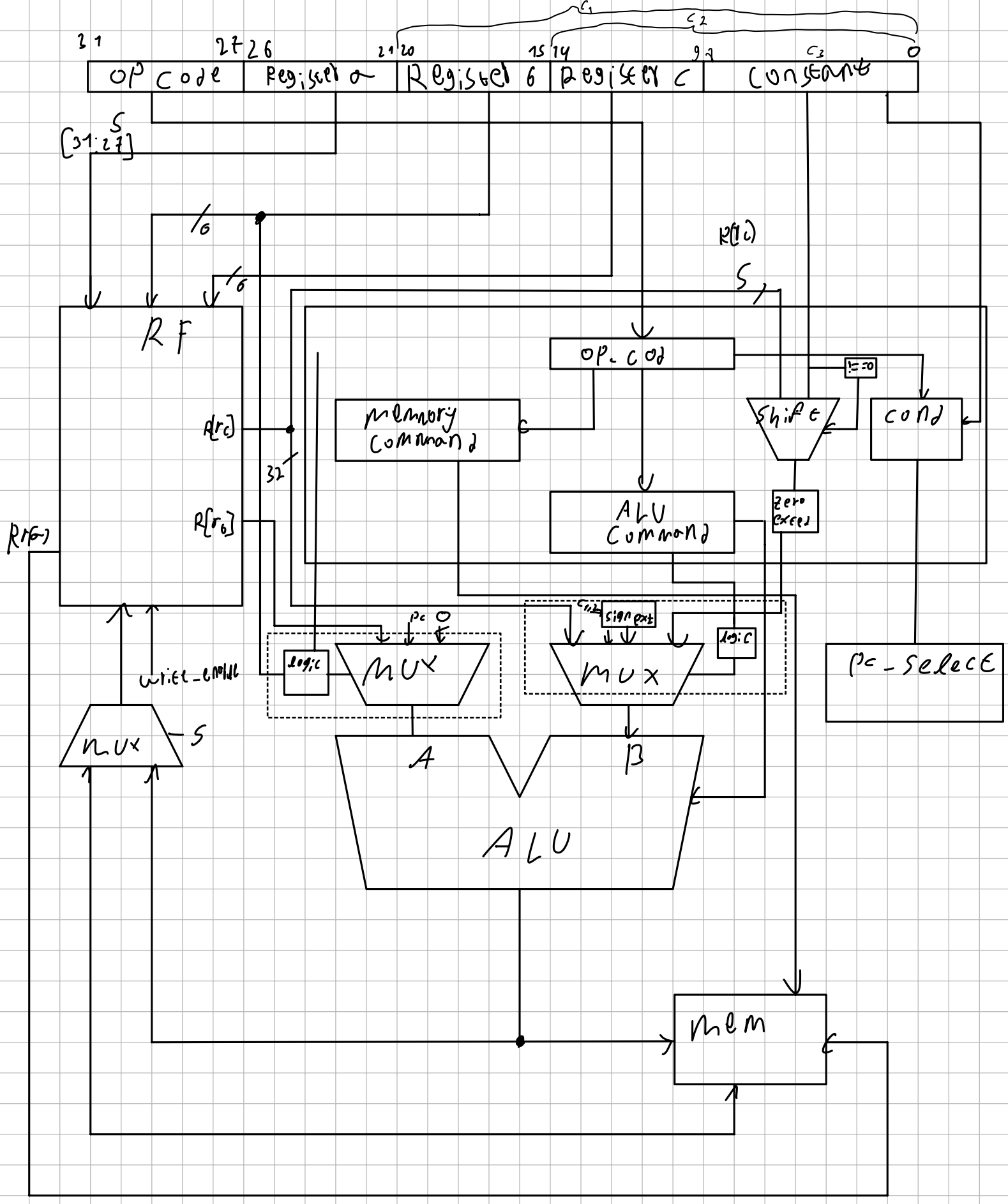

Pc_Split
Incu_Split
Incu_in
Incu
Incu_out
mux_in_00
muxs_wire_connect
Regiser_connect
branc_out
reg_mux_in_0
reg_mux_in_1
Branch in
branch
reg
reg mux
reg_mux_out
mux_in01
Pc_out
conse_in
reg_mux_select
pc_mux
pc_mux_out
nexe_pc
DFF
Pc_out
Pc_out
clk
reset
stay
constant in
sign exted
reg_check_selece
mux_in_10
conse_out
branch_wire
mux_in_11
2
select
conse_Split
reg_selece_wire
constant
select
clock
reset
stay
16 bit
1 bit


Pc_Split
Incu_Split
Incu_in
Incu
Incu_out
mux_in_00
branc_out
neg_mux_in_0
Neg_mux_in_1
Branch in
branch
neg
neg_mux
neg_mux_out
mux_in_01
Pc_out
conse_in
neg_mux_select
neg_check_select
mux_in_10
constant in
sign extend
conse_out
branch_wire
mux_in_11
neg_selece_wire
conse_Split
muxs_will_connect
Regiseel_connect
PC_mux
2
select
PC_mux_out
next_pc
DFF
clk
rese
stay
PC_register_out
Pc_Out
constant
select
clock
rese
stay
16 bit
1 bit


Pc_Split
Incu_Split
Incu_in
Incu
Incu_out
mux_in_00
branc_out
reg_mux_in_0
reg_mux_in_1
Branch in
branch
reg
reg mux
reg_mux_out
mux_in_01
Pc_out
conse_in
reg_mux_select
constant_in
sign exted
sign_connect
reg_check_selece
mux_in_10
[15:3]
conse_out
branch_wire
[15:0]
mux_in_11
conse_split
reg_selece_wire
muxs_will_connect
PC_mux
2
select
Regiseer_connect
PC_mux_out
nexe_pc
DFF
clk
rese
stay
PC_register_out
Pc_Out
constant
select
clock
rese
stay
16 bit
1 bit


memory[PC_out]
PC_out
command
op_code
control
Registers
RF
const
sign extn
ALU, PC


command
op code
Register a
Register b
Register c
constants
Register b check
RF
control
const_block
ALU
signals
const register
const extension
RF



logic_1 : NCg, noe 0 0 → 1

ld, SE, la, rb!=0 0 → A

LPR, STR, LAR pc → A

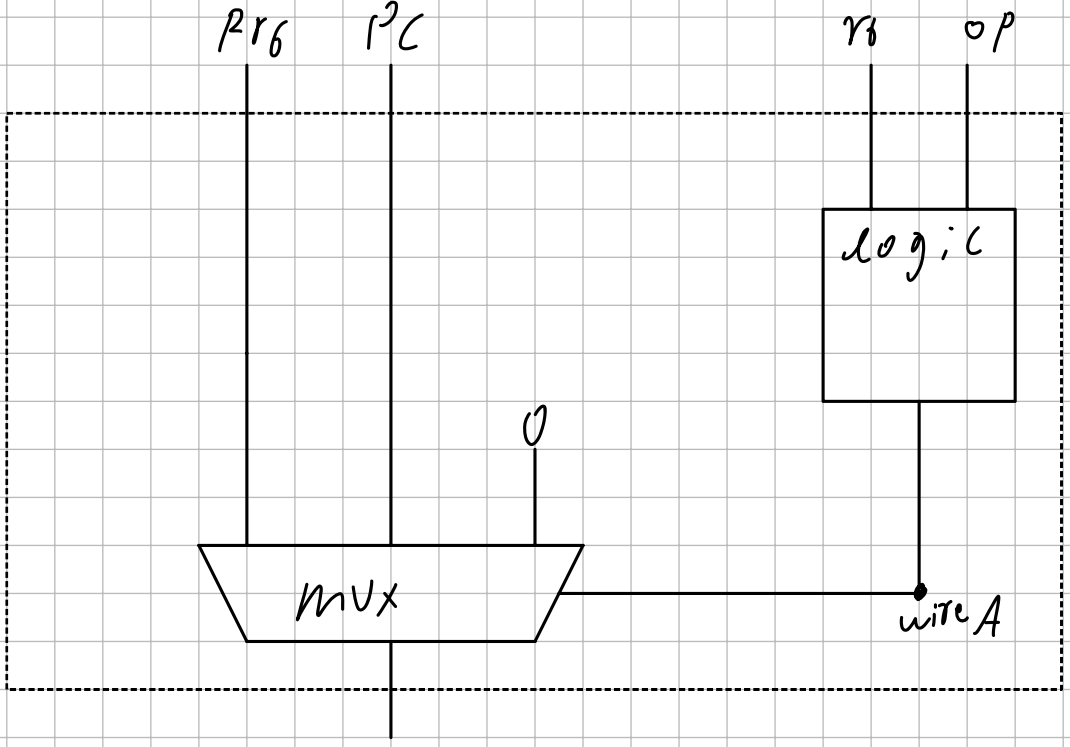
lsle R[rb] → A

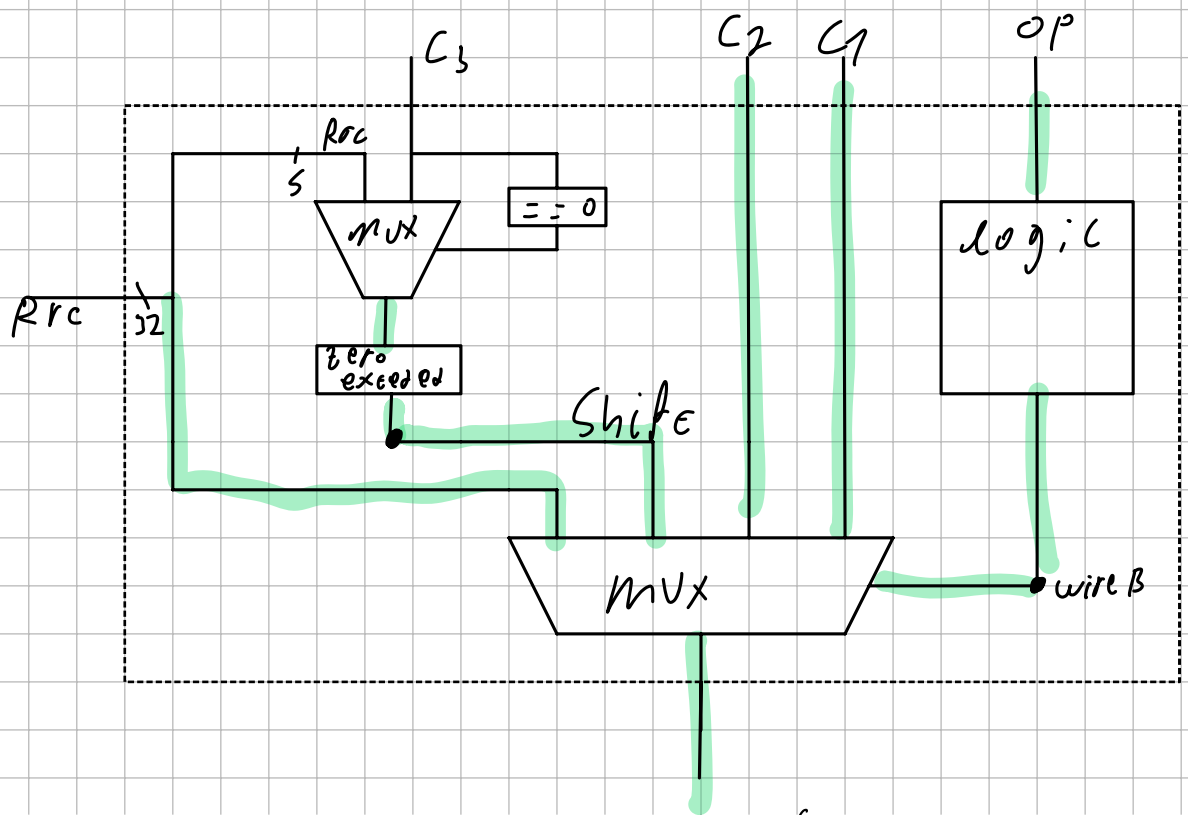
Logic_B: ld, se, la, addi, andi, ori, xori, LTI, EQI $C_2 \rightarrow B$

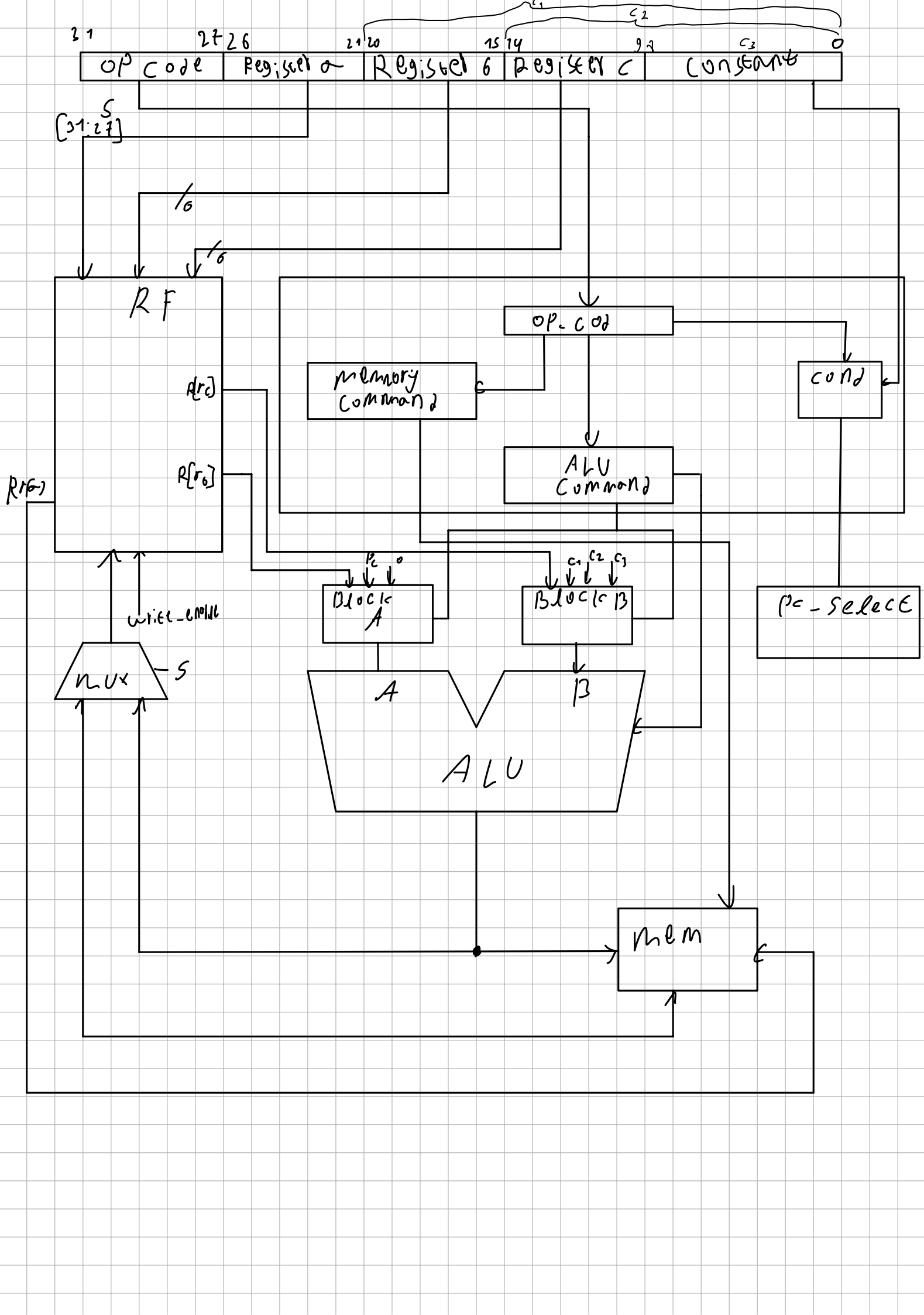
Shift Shift $\rightarrow B$

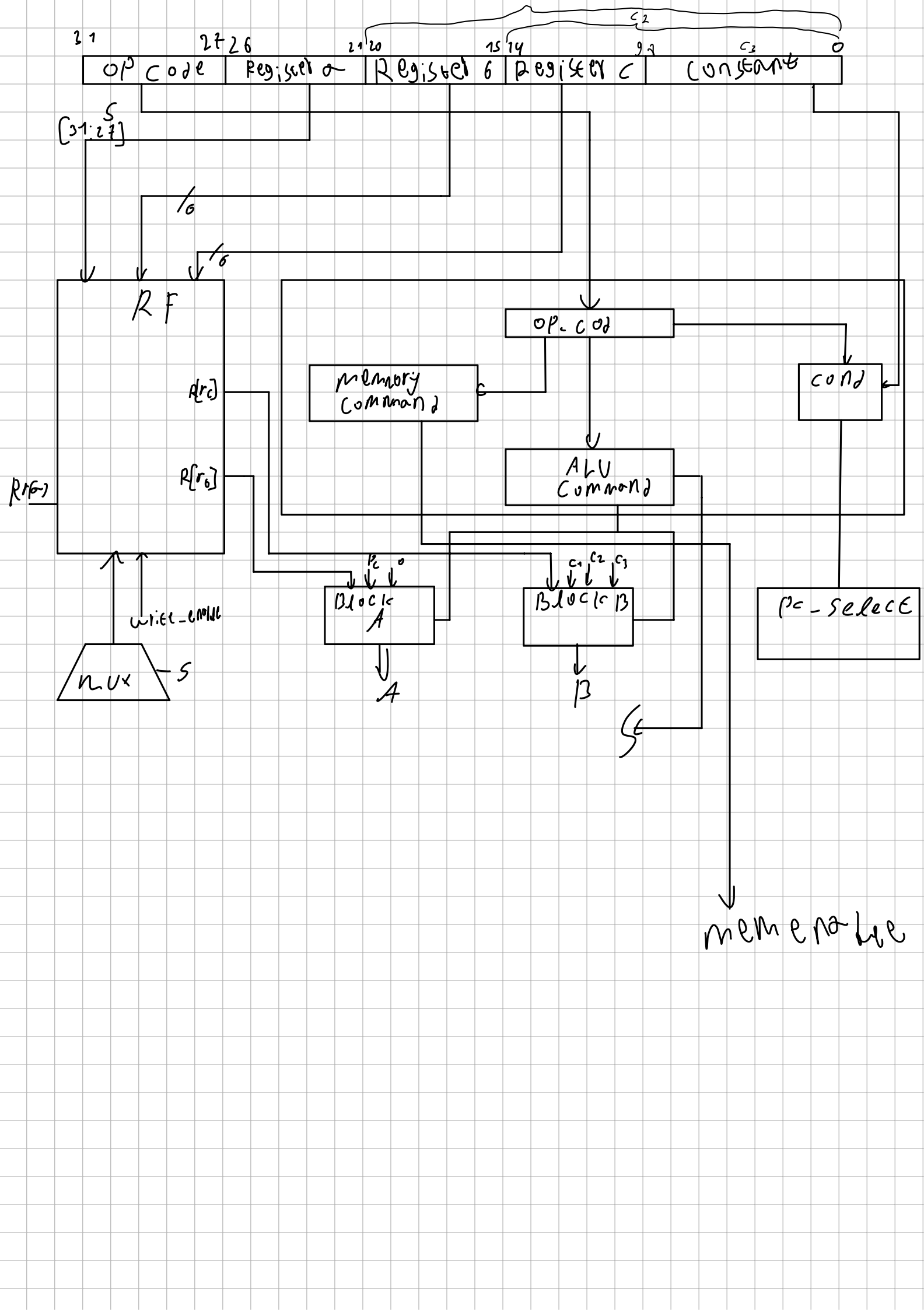
LDF, STR, LHR $C_1 \rightarrow B$

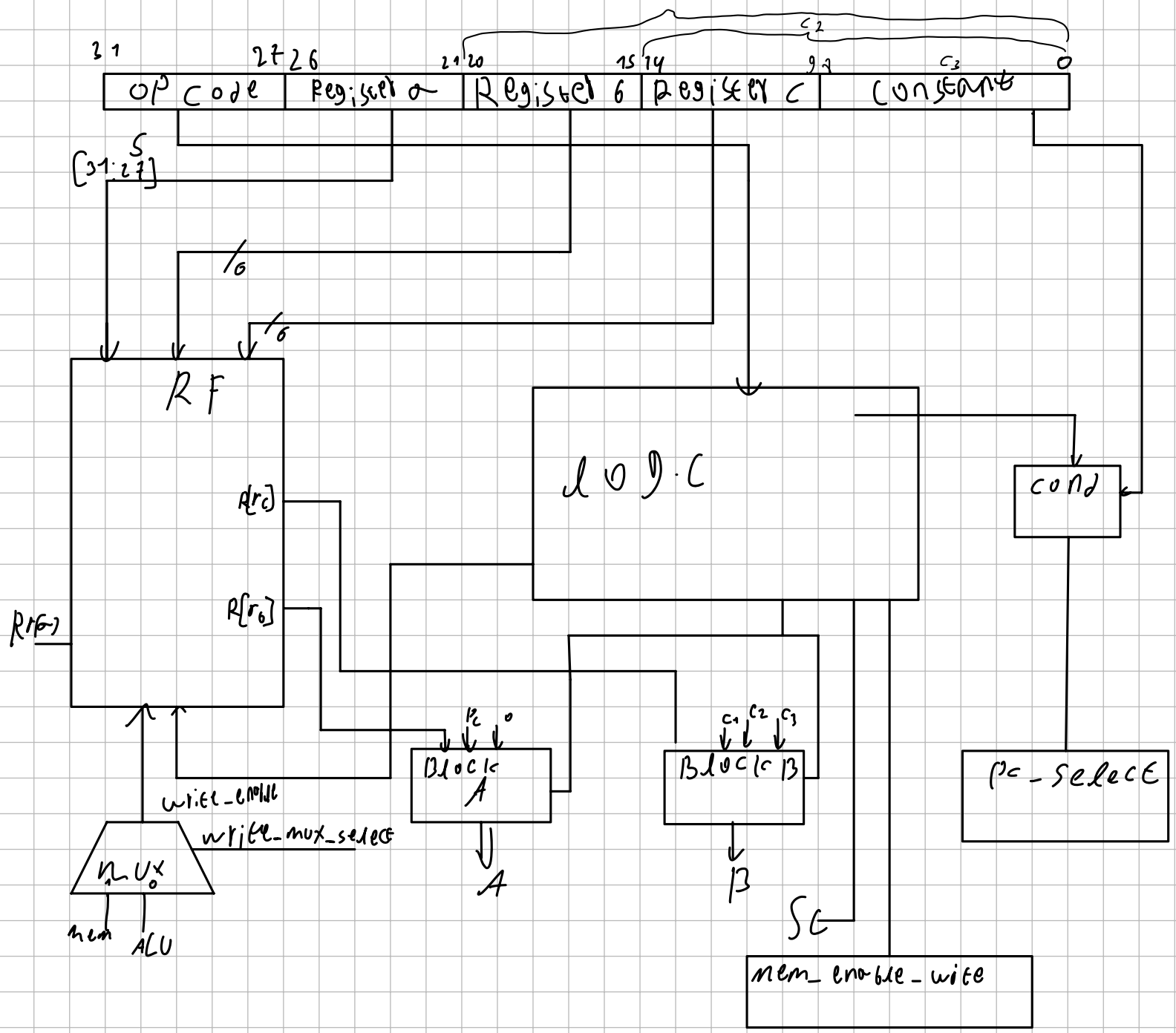
else $R[rc] \rightarrow B$






31
27 26
21 20
15 14
9 8
0
c1
c2
c3
op code
Register a
Register b
Register c
constant
5
[31:27]
6
6
RF
op_cod
memory command
cond
ALU Command
A[rc]
R[rb]
R[rb]
Pc
0
c1
c2
c3
Block A
Block B
pc_select
write_enable
mux
5
A
B
ALU
mem


31
27 26
21 20
15 14
9 8
0
C2
C3
Op code
Register a
Register b
Register c
Constante
5
[31:27]
6
6
6
RF
Op. cod
memory command
cond
ALU Command
A[rc]
R[rb]
R[ra]
Pc
0
C1
C2
C3
Block A
Block B
Pc_selecE
write_enable
mux
5
A
B
S
memenable


31
27 26
21 20
15 14
9 8
0
c2
c3
OP code
Register a
Register b
Register c
constante
5
[31:27]
6
6
6
RF
A[ra]
B[rb]
RWE
write_enable
write_mux_select
MUX
mem
ALU
LOAD
cond
Block A
Block B
A
B
SC
mem_enable_write
pc_select

PC	mux_write_sel	mem write enable	RF write enable	ALU OP	OP code
+4	ALU	0	1	ADD	OP_ADD
+4	ALU	0	1	ADD	OP_ADDI
+4	ALU	0	1	SUB	OP_SUB
+4	ALU	0	1	SUB	OP_NEG
+4	ALU	0	1	NOT	OP_NOT
+4	ALU	0	1	AND	OP_AND
+4	ALU	0	1	AND	OP_ANDI
+4	ALU	0	1	OR	OP_OR
+4	ALU	0	1	ORI	OP_ORI
+4	ALU	0	1	XOR	OP_XOR
+4	ALU	0	1	XORI	OP_XORI
+4	ALU	0	1	SL	OP_SL
+4	ALU	0	1	SR	OP_SR
+4	ALU	0	1	SRS	OP_SRS
+4	ALU	0	1	SC	OP_SC
+4	ALU	0	1	LT	OP_LT
+4	ALU	0	1	LT	OP_LTI
+4	ALU	0	1	EQ	OP_EQ
+4	ALU	0	1	EQ	OP_EQI
+4	ALU	0	1	ADD	OP_LA
+4	ALU	0	1	ADD	OP_LAR
+4	mem	0	1	ADD	OP_LD
+4	mem	0	1	ADD	OP_LDR
+4	X	1	0	ADD	OP_ST
+4	X	1	0	ADD	OP_STR
cond→Branch else +4	X	0	0	X	OP_BRANCH
Jump	X	0	0	X	OP_JUMP

ALU = 0
mem = 1
X = X

Branch

C3 = 1

225 · 1024 = 230400 byte

230400/4 = 57600 words

PC:
instruction_mem: 2^{16} = 65536 byte

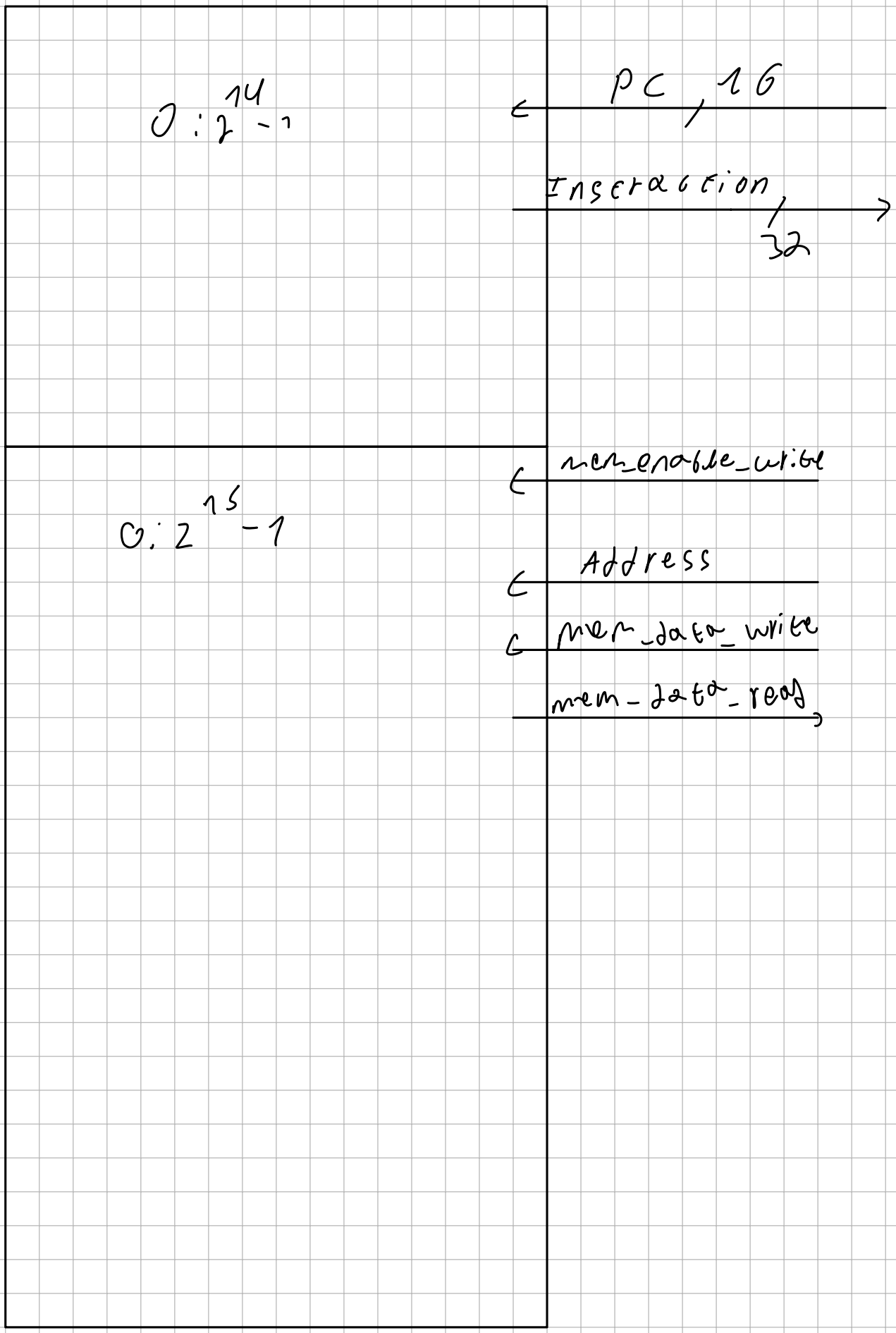
$2^{16}/2^2$ = 16384 words

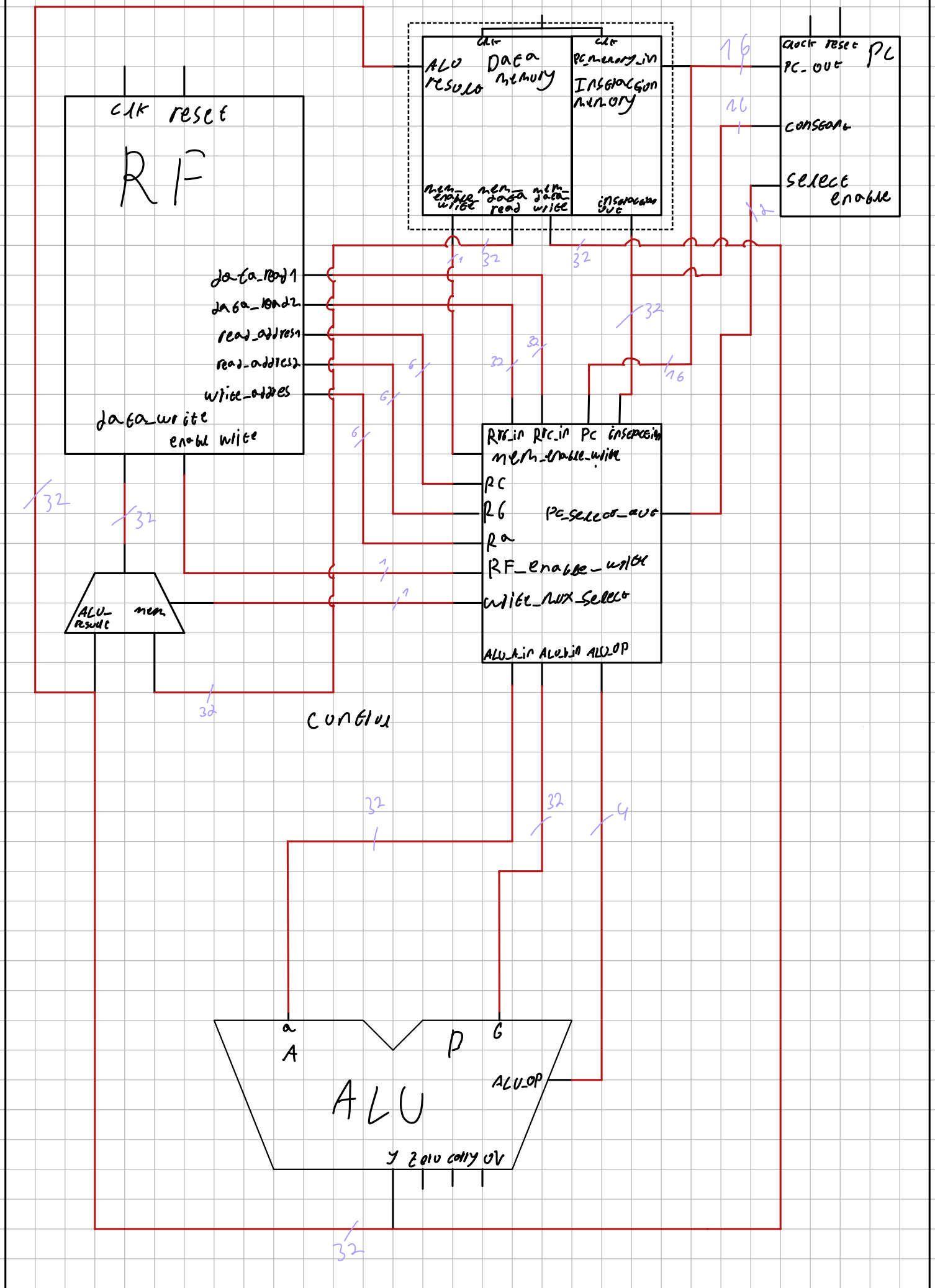
left 57600 - 16384 = 41216

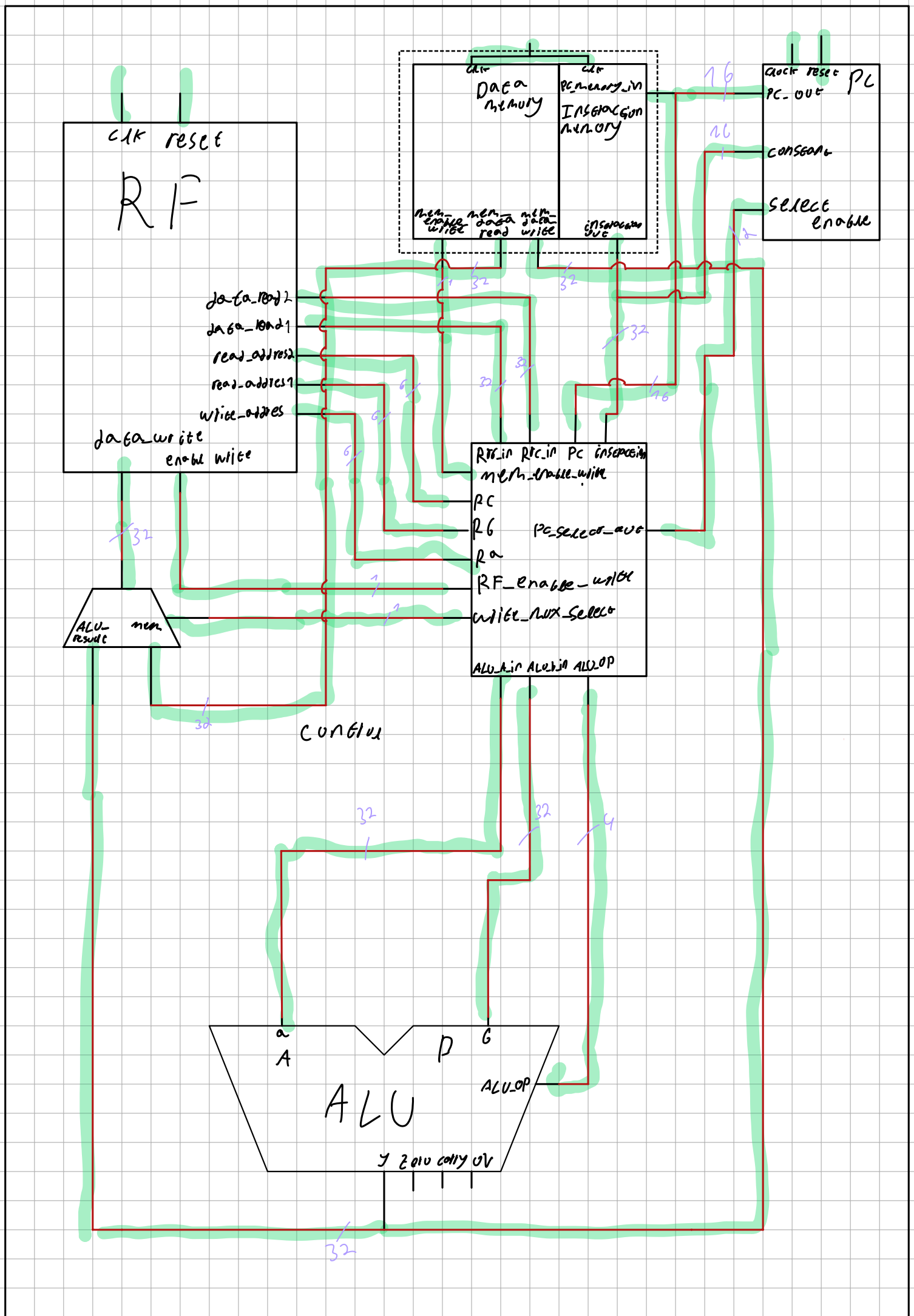
memory data size = 2^{15} = 32768 words

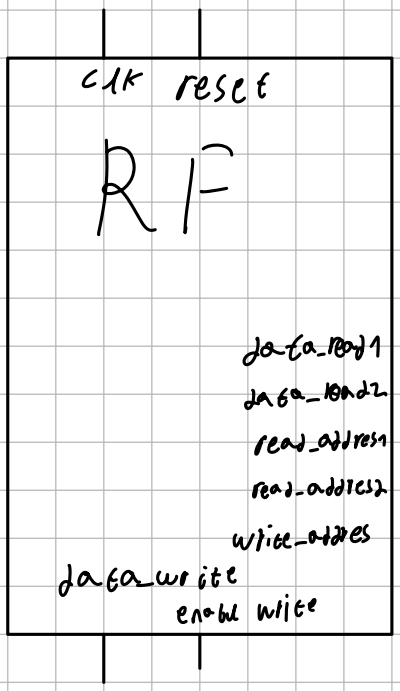
left 8448 words

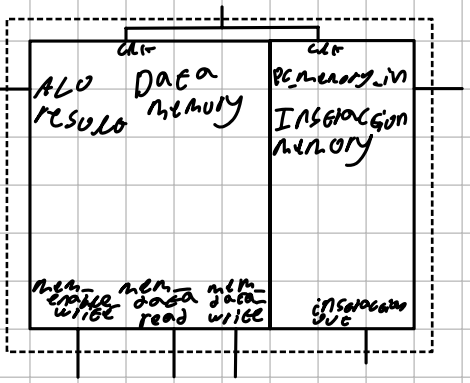
Instruction_memory

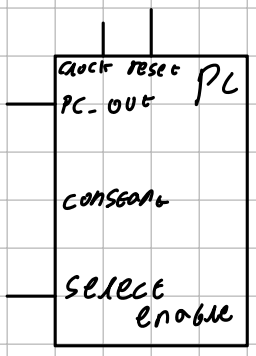


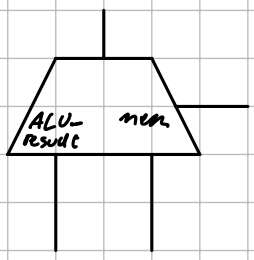

clk reset
RF
data_read1
data_read2
read_address1
read_address2
write_addres
data_write
enable write
clk
ALU result
Data memory
clk
pc_memory_in
Instruction memory
mem_enable_write
mem_data_read
mem_data_write
instruction out
clock reset
PC
PC_out
consGen
select enable
RR_in RC_in Pc instruction
mem_enable_write
PC
R6
Ra
pc_select_out
RF_enable_write
write_mux_select
ALU_A_in ALU_B_in ALU_OP
ALU_result
mem
CONTROL
a
A
B
P
ALU
ALU_OP
Y Zero Carry OV

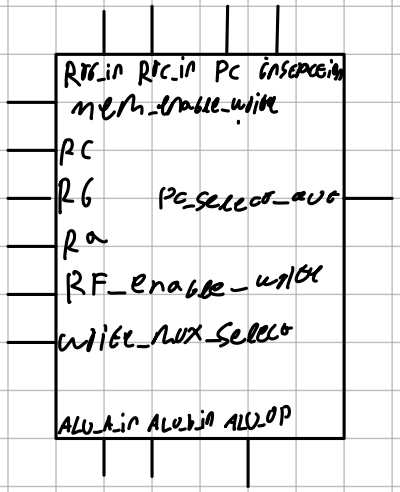

clk
reset
RF
data_read2
data_read1
read_address2
read_address1
write_addres
data_write
enable write
ALU_ result
mem
32
clk
clk
Data memory
Instruction memory
pc_memory_in
mem_ enable write
mem_ data read
mem_ data write
instruction out
clock
reset
PC
PC_out
16
consGen
select enable
RF_in
RF_in
Pc
instruction
mem_enable_write
PC
RB
Ra
PC_select_out
RF_enable_write
write_mux_select
ALU_A_in
ALU_B_in
ALU_OP
CONTROL
32
4
a
A
B
ALU
ALU_OP
Y
Zero
Carry
OV


clk
reset
RF
data_read1
data_read2
read_address1
read_address2
write_addres
data_write
enable write

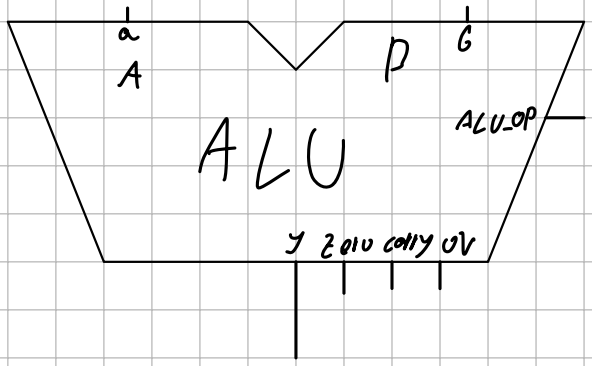

clk
clk
ALU result
Data memory
PC_memory_in
Instruction memory
mem_enable_write
mem_data_read
mem_data_write
instruction out


clock
reset
PC
PC_out
consGen
select
enable


ALU_result
mem


RF_in
RF_in
PC
instruction
mem_enable_write
PC
R6
Ra
RF_enable_write
write_mux_select
PC_select_out
ALU_A_in
ALU_B_in
ALU_OP

CONTROL


a
A
B
ALU
ALU_OP
Y
Zout
Cout
V